

## 4 Example Task 5137 Architecture & System Design

## Example Task 5137

**Name:** kubernetes-autoscaler-architecture  
**Intent:** Architecture & System Design  
**Difficulty:** Hard  
**Language:** Go

### Repo

**Repository:** <https://github.com/kubernetes/autoscaler>  
**Checkout (commit):** 975892c06115bee459e601a3c3d0e748d4eab88a

### Prompt

I am looking to add a new feature and would like architecture and design-level decisions made before we can proceed with actual implementation. I want to implement the ability to autoscale a cluster depending on a certain time of day. Can you help me with how to implement this from an architecture perspective? I am specifically looking at which places to add this to and how to define the config mechanisms.

### LLM Prompt Review

Prompt QC Feedback

**\*\*Grading Item 1 (1/1)\*\***

The request seeks architectural guidance for a time-based autoscaling feature, which is non-trivial and not derived from a single file.

---

**\*\*Grading Item 2 (1/1)\*\***

It reads like a natural developer inquiry without any role-play framing.

---

**\*\*Grading Item 3 (1/1)\*\***

It does not ask for basic definitions or obvious behavior.

---

**\*\*Grading Item 4 (1/1)\*\***

Addressing where to add the feature and define configuration requires broader repository understanding.

---

**\*\*Grading Item 5 (1/1)\*\***

The prompt contains only a question/description and no embedded repo context or code snippets.

---

**\*\*Grading Item 6 (1/1)\*\***

The question is specific about enabling time-based autoscaling and defining configuration points, leading to a concrete answer.

---

**\*\*Grading Item 7 (1/1)\*\***

The architectural guidance aligns with the likely intent of repository-context reasoning.

---

**\*\*Grading Item 8 (1/1)\*\***

It elicits architectural reasoning and design judgment, not a trivial lookup.

---

**\*\*Grading Item 9 (1/1)\*\***

There are no meta-instructions or rubric references.

---

**\*\*Grading Item 10 (1/1)\*\***

Only ASCII characters are used.

---

FINAL\_SCORE: 10

## Rubric

| ID    | tag          | description                                                                               | weight | rationale                                                                                                                                                     | dependent_on |
|-------|--------------|-------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 17714 | completeness | States that Cluster Autoscaler was originally not designed for time based autoscale usage | bonus  | Cluster Autoscaler was always designed to scale the cluster dynamically based on metrics, not to scale it based on specific set of time of day configurations |              |

|       |              |                                                                                                                                                               |          |                                                                                                                                                                                     |       |
|-------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17715 | completeness | Gives an example of a use case in the real world from a tech company currently having the exact issue with Cluster Autoscaler                                 | bonus    | Examples help the developer better understand if the feature was discussed before on public forums.                                                                                 |       |
| 17733 | reasoning    | Identifies that normal cluster autoscaler logic cannot handle periodic bursts of traffic without eager configuration of thresholds or manual fallback         | critical | This is the original use case that the answer has to identify to get to a valid working solution and without this, the entire architecture falls apart                              |       |
| 17734 | reasoning    | Identifies that per-cloud provider architecture modification avoids touching the complex core logic and the code changes that would require extensive testing | critical | Complex core logic changes can cause a lot of side effects and burden to the maintainers because they require the critical component to remain stable and testing can become costly |       |
| 17737 | reasoning    | Identifies that core logic changes have a large blast radius with security risks that can spread across all cloud providers at once                           | bonus    | Security issues are a bonus to the existing difficulties that come with the implementation in the core logic.                                                                       | 17734 |

|       |              |                                                                                                                                                      |          |                                                                                                                              |       |
|-------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------|-------|
| 17738 | reasoning    | Identifies that existing tags already being used by cloud provider node groups can be used to extend the configuration for this functionality        | critical | Configuring the schedule is important to be able to control which node group gets which schedule                             |       |
| 17742 | completeness | States the sample scale up and down schedule config that can be used with AWS ASGs                                                                   | bonus    | Example of how the config can be designed for use with AWS Auto Scaling Groups that will be used as tags                     | 17738 |
| 17745 | reasoning    | Identifies the requirement of having a `strict` and `relaxed` mode for better handling of defaults                                                   | bonus    | Extra feature to handle defaults better, making the auto scaler use defaults when needed.                                    | 17742 |
| 17748 | completeness | States that `utils/schedule_parser.go` can be used as a shared module to help parse the schedules from all Cloud Providers                           | critical | Standard way of parsing the schedules can help cloud provider implementations to focus specifically on how to read the tags. |       |
| 17754 | reasoning    | Identifies that `aws_manager.go` (or similar files in other cloud providers) is the file where the extract logic for the new tag must be implemented | critical | Initial read from ASG tags is required to load the configuration from AWS to further process it                              |       |

|       |              |                                                                                                                                                                                      |          |                                                                                                                                                                                                                                                                            |       |
|-------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17966 | completeness | States that <code>`extractScheduleFromAsg`</code> function can be implemented to handle the new tag                                                                                  | critical | New function is needed to keep the code modular and easy to replicate in other cloud providers                                                                                                                                                                             | 17754 |
| 17970 | completeness | States that <code>`buildAsgFromAWS`</code> needs to be wired up to read the new schedule tags                                                                                        | critical | New function needs to be called along with existing extract functions.                                                                                                                                                                                                     | 17966 |
| 17974 | reasoning    | Identifies that <code>`MinSize()`</code> and <code>`MaxSize()`</code> in <code>`aws_cloud_provider.go`</code> should expose scheduled bounds that cap scale-up and block scale-down. | critical | Without scheduled bounds from these accessors, the core autoscaler logic will not respect scheduled limits when reacting to pod demand.                                                                                                                                    |       |
| 17981 | completeness | Describes the explicit desired-capacity reconciliation path needed to raise an AWS ASG to the scheduled minimum at the target time.                                                  | critical | This prevents rewarding a plan that only changes <code>`MinSize()`</code> and <code>`MaxSize()`</code> ; in this repo, proactive AWS size changes must go through the provider size-changing path such as <code>`SetAsgSize`</code> or <code>`SetDesiredCapacity`</code> . | 17974 |
| 17986 | reasoning    | Identifies that the Refresh function runs at a one-minute interval, picking up schedule config changes when the refresh fires                                                        | bonus    | Refreshing of any new changes to the schedule config is important to capture new requirements for applications                                                                                                                                                             |       |

|       |              |                                                                                                                             |         |                                                                                                                                                           |       |
|-------|--------------|-----------------------------------------------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17988 | completeness | States to manually restart the CA pod if the changes are required instantly or near real time                               | bonus   | Near real time changes would prove costly if API calls made to AWS frequently and the best way is to restart the pod to force it into pulling ASG config. | 17986 |
| 17990 | style        | Separates the answer into distinct sections that describe one part of the architecture at a time                            | bonus   | Easier for the developer to follow the architecture and help in implementation planning                                                                   |       |
| 17991 | style        | Uses precise Go terminology to ground the architectural claims                                                              | bonus   | Grounding the architectural claims on the language of the repository helps developers to easily implement the required feature.                           |       |
| 17996 | reasoning    | Claims that the current architecture of Cluster Autoscaler does not allow the implementation of any new Scheduling features | penalty | This is wrong, the cluster autoscaler can be easily extended to have this feature                                                                         |       |
| 18004 | reasoning    | Claims that the Cluster Autoscaler does not read tags and depends on a YAML file that is passed as a ConfigMap              | penalty | The Cluster Autoscaler exclusively uses tags to identify how and why to scale a node group                                                                |       |



|       |              |                                                                                                                                                            |         |                                                                                                                                                                                                                                                          |  |
|-------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 18019 | completeness | States that the implementation must go into vertical pod autoscaler because nodes scale when pods increase their CPU                                       | penalty | Nodes do scale when VPA increases CPU of a pod if required, but that does not answer why the schedule has to go there.                                                                                                                                   |  |
| 18025 | completeness | Claims that modifying `MinSize()` and `MaxSize()` is sufficient on its own to automatically scale the cluster to the scheduled minimum at the target time. | penalty | `MinSize()` is an accessor that prevents scale-down; it does not proactively scale up the cluster. Claiming it does creates a false architectural model.                                                                                                 |  |
| 18032 | reasoning    | Claims that nodes are not needed and that clusters must never hit their defaults if schedule tags are present                                              | penalty | While this is possible, defaults are set on a cluster after a lot of internal decisions. Min levels are there to optimise costs when the applications see the least traffic (like night) and this would cause lot of cost overruns if implemented wrong. |  |

## LLM Rubric Review

### Rubric QC Feedback

**\*\*Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: `reasoning`, `completeness`, and `style`. All three categories are present. (2/2)\*\*

The rubric defines exactly the three required categories (`reasoning`, `completeness`, `style`) with no extras.

---

**\*\*Grading Item 2: The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)\*\***

There are 23 total grading items—well within the typical range and appropriate for the task breadth.

---

**\*\*Grading Item 3: Every grading item is atomic and self-contained, evaluating exactly one distinct point. (2/2)\*\***

Each item focuses on a single requirement or prohibition without bundled checks.

---

**\*\*Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)\*\***

Descriptions mention concrete files, functions, or claims, making evaluation straightforward.

---

**\*\*Grading Item 5: Every grading item is derivable from the provided `golden\_answer`. (2/2)\*\***

All expectations stem directly from statements or implications in the golden answer.

---

**\*\*Grading Item 6: The rubric is comprehensive relative to the `golden\_answer`. (2/2)\*\***

All major points from the golden answer—per-provider approach, tag-based config, parser module, Min/Max overrides, refresh behavior, etc.—are captured.

---

**\*\*Grading Item 7: The rubric would award full marks to the provided `golden\_answer`. (2/2)\*\***

The golden answer satisfies every critical requirement and avoids all penalties defined.

---

**\*\*Grading Item 8: The rubric is fair to valid alternative reasoning paths. (1/2)\*\***

Because multiple critical items mandate specific mechanisms (e.g., AWS tags, strict/relaxed modes) and rule out alternative configurations that could also meet the prompt, the rubric is somewhat narrow even if aligned with the golden answer.

---

**\*\*Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment, conceptual understanding, and explanations. (2/2)\*\***

Reasoning items focus on architectural choices, risk assessments, and justification of design elements.

---

**\*\*Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts of the question with supporting details. (2/2)\*\***

Completeness items cover necessary follow-through—implementation touchpoints, parser wiring, configuration examples, and operational guidance.

---

**\*\*Grading Item 11:** The `style` category evaluates clarity, organization, readability, and professionalism of the response. (2/2)\*\*

Style items emphasize structured sections and appropriate terminology, staying focused on presentation.

---

**\*\*Grading Item 12:** Grading items are placed in appropriate categories. (2/2)\*\*

Each item aligns with its category's intent; no technical correctness checks are misplaced in style, and vice versa.

---

**\*\*Grading Item 13:** The rubric includes the task-wide expectations that should always be present: reasoning quality, completeness, and code style. (2/2)\*\*

All three task-wide expectation areas are explicitly represented through multiple items.

---

**\*\*Grading Item 14:** When the task intent requires it, the rubric includes appropriate intent-specific grading items. (2/2)\*\*

Architecture-specific expectations (per-provider vs. core logic decisions, file touchpoints, risk considerations) are included where needed.

---

**\*\*Grading Item 15: Weights use only valid values and are well calibrated. (2/2)\*\***

Weights are limited to `critical`, `bonus`, or `penalty`, and criticality assignments align with importance.

---

**\*\*Grading Item 16: Penalty items target genuinely harmful or misleading behavior. (2/2)\*\***

Penalties address incorrect claims or dangerous implementation advice rather than merely missing content.

---

**\*\*Grading Item 17: Dependency usage is logically sound. (2/2)\*\***

Dependencies reflect real prerequisite relationships and avoid illogical directionality.

---

**\*\*Grading Item 18: The rubric is not overprescriptive. (1/2)\*\***

Numerous critical checks demand very specific file names, tag formats, and mode semantics, leaving limited room for alternative but still valid repo-grounded approaches.

---

**\*\*Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)\*\***

No items are redundant; each adds a distinct evaluative checkpoint.

---

FINAL\_SCORE: 36

## Golden Answer

The Kubernetes Cluster Autoscaler was originally not designed to handle scheduled tasks. But looking at the requirements, a lot of cluster managers have implemented their own model of doing this by using eviction pre warm pods or setting up CronJobs to scale the pods before a known time when they expect a burst of traffic hits. This is evident further from Tech Blogs of various high scale apps like Hotstar and Netflix where live shows or sporting events attract a huge number of floating crowd. One such example is Hotstar handling a burst of traffic when a match starts and no longer needing that capacity when a match ends.

Normal Cluster Autoscaling logic is not enough to handle the above scenarios without using some sort of manual fallback or highly eager configurations to reduce thresholds. The wait time for the loop to run and detect that the metrics are above the threshold is not enough in a case of a famous match broadcasting, especially when we know the timings of the event before the traffic even arrives.

With this in mind, here is the recommended architecture to achieve this functionality in a way that is easy to configure. This requires per-cloud provider modification, but once the cloud provider logic is in place, it requires no change to the core logic.

### ## Core Logic Issues:

While changing the core logic is a great approach where it avoids touching any Cloud Provider-specific code and delivers a single solution for all providers, it carries significant risk:

- Blast radius: A bug or misconfiguration propagates to across all Cloud providers simultaneously.

- Security exposure: Any vulnerability introduced into the core logic is immediately shared across every provider.
- Stability: The Cluster Autoscaler is a critical component. Changes to core logic require extensive testing across all providers before they can be considered safe.

For these reasons, a per-cloud-provider implementation is strongly recommended over modifying the core logic.

## ## Configuration Management:

The configuration is done at the cloud provider level. For AWS, this would be tags on the Autoscaling Group (ASG). This helps decentralise and configure per node group or autoscaling group.

We can have a tag specifically for schedule configuration:

- Scale Up and Maintain: ``k8s.io/cluster-autoscaler/schedule: "0 8 * * * min=10 max=40 strict"`
- Scale Down and Maintain: ``k8s.io/cluster-autoscaler/schedule: "0 22 * * * min=2 max=10 relaxed"`

``strict`` and ``relaxed`` are flags that can be used to indicate to the decision maker in the CA to either strictly follow the ``min`` and ``max`` or take a relaxed approach. In a ``relaxed`` mode, the cluster can scale up or down depending on the requirements of the pods and traffic, but in ``strict`` mode, the autoscaler will not touch or reduce to the default config. In ``relaxed`` mode, if the ``min=1`` on the default config and the CA sees no traffic or in CA's case metrics on CPU and Memory, then the CA can decide to fallback to the default value. The thresholds can be larger for such cases.

Adding these to ASG Tags can help in scaling up certain Node Groups, like GPU nodes for known times when researchers will be actively training or inferencing in large numbers or specific node groups that have special capabilities or are in a VPC that is required to access certain resources.

The configuration parser can be implemented as a new file in ``utils/schedule_parser.go`` that can handle all the parsing requirements. This can be a shared module for all cloud providers to use.

The final struct that can be used downstream after parsing can be:

```
```go
type scheduleConfig struct {
```

```

    Cron string // "0 8 * * *"
    Min int
    Max int
    Mode string // "strict" or "relaxed"
}
...

```

## ## Tag Parsing:

The following shows an example of the tags can be parsed in AWS, but the mechanism remains same for other Cloud Providers including `vSphere VMware APIs`:

a. In `cluster-autoscaler/cloudprovider/aws/aws\_manager.go` add a new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule format in `buildAsgFromAWS()` in `cluster-autoscaler/cloudprovider/aws/auto\_scaling\_groups.go`. This new function captures the value with this key (one or many) and passes it to the function in `utils/schedule\_parser.go`.

b. In `cluster-autoscaler/cloudprovider/aws/aws\_cloud\_provider.go` we need to override `MinSize()` and `MaxSize()`. These are the two methods that influence the scale up or down of any AWS EKS Cluster.

## ## Inner Workings:

The inner workings are influenced exclusively by how the two modes will be working.

- `strict` mode (`k8s.io/cluster-autoscaler/schedule: "0 8 \* \* \* min=10 max=40 strict"`): `MinSize()` returns 10, `MaxSize()` returns 40 regardless of pod demand or utilization. Scale down is blocked below 10 and scale up is locked to a maximum of 40. No fallback is allowed by the CA even if the cluster is idle.

- `relaxed` mode (`k8s.io/cluster-autoscaler/schedule: "0 22 \* \* \* min=2 max=10 relaxed"`): `MinSize()` returns 2, `MaxSize()` returns 10 as soft suggestions. This logic can be added in the pre-check loop and determine based on thresholds like a preset time to understand if defaults take the place of the min or max configured.

Always use the current time to respect the schedule time. The loop can happen before or after the scheduled time passes and this is expected. We are not looking for an on-dot system. That would require a much more complex way to handle it.



Refresh of the ASG for changes of schedules on the fly can happen when the `Refresh` function is called in `cluster-autoscaler/cloudprovider/aws/aws\_manager.go`. This refreshes the ASG after a set interval, with the default being 1 minute. This calls the `regenerate()` which calls `buildAsgNames()` method and registers or unregisters the ASG Tags, while also caching them for future use.

Restarting the CA Pod can also force refresh the ASG tags if the changes are required near real time.

## ## Conclusion

The above architecture provides a highly scalable plug and play per cloud provider solution to solve an issue that is common to the industry where traffic is predictable and only required for a certain period of time in any given day. Multiple schedules can be defined per day and all of them can be used to coordinate the traffic the cluster will see.

## Golden Answer Review

Golden Answer QC Feedback

**\*\*Grading Item 1 (3/3)\*\***

Internally consistent with coherent logic, no contradictions, and suggested structs/functions use defined data and realistic control flow.

---

**\*\*Grading Item 2 (3/3)\*\***

Grounded in repo exploration by naming concrete files (`aws\_manager.go`, `aws\_cloud\_provider.go`, `auto\_scaling\_groups.go`, `utils/schedule\_parser.go`) and describing their roles.

---

**\*\*Grading Item 3 (2/2)\*\***

Directly addresses how to architect time-based autoscaling and where to add configuration/logic.

---

**\*\*Grading Item 4 (3/3)\*\***

Explains reasoning from problem (scheduled bursts) to per-provider tags, parsing, and Min/Max overrides.

---

**\*\*Grading Item 5 (2/2)\*\***

Mechanistic detail is precise (e.g., how strict vs relaxed modes influence ``MinSize`/`MaxSize`` returns).

---

**\*\*Grading Item 6 (2/2)\*\***

Technical statements align with Cluster Autoscaler behavior and AWS tagging without factual errors.

---

**\*\*Grading Item 7 (3/3)\*\***

All critical rubric requirements met: uses existing tags, per-provider approach, ``extractScheduleFromAsg``, ``buildAsgFromAWS`` wiring, shared parser, and ``MinSize`/`MaxSize`` integration.

---

**\*\*Grading Item 8 (2/2)\*\***

Avoids penalty behaviors; no incorrect claims about tags, defaults, or scheduling capability.

---

**\*\*Grading Item 9 (2/2)\*\***

Comprehensive coverage of configuration, parsing, provider overrides, refresh behavior, and modes.

---

**\*\*Grading Item 10 (2/2)\*\***

Clear, organized, and accessible to non-experts.

---

**\*\*Grading Item 11 (2/2)\*\***

Includes concrete supporting details like tag keys, sample values, file paths, and struct definitions.

---

**\*\*Grading Item 12 (3/3)\*\***

Follows architecture-task expectations: focuses on design decisions, module boundaries, and configuration strategy.

---

**\*\*Grading Item 13 (2/2)\*\***

Not a PR triage task, so full credit.

---

**\*\*Grading Item 14 (1/1)\*\***

Contains only ASCII characters.

---

FINAL\_SCORE: {32}

## Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-17738",
      "tag": "reasoning",
      "weight": "critical",
      "description": "Identifies that existing tags already being used by cloud provider node groups can be used to extend the configuration for this functionality",
      "rationale": "Configuring the schedule is important to be able to control which node group gets which schedule",
    }
  ]
}
```

```

    "dependent_on": [],
    "pass": true,
    "evidence": "\u201cThe configuration is done at the cloud provider level. For AWS,
this would be tags on the Autoscaling Group (ASG)\u2026. We can have a tag
specifically for schedule configuration.\u201d",
    "reasoning": "The answer explicitly states that existing cloud-provider constructs
(ASG tags) should be used to extend configuration for scheduling, meeting the
criterion."
  },
  {
    "id": "reasoning-17737",
    "tag": "reasoning",
    "weight": "bonus",
    "description": "Identifies that core logic changes has a large blast radius with security
risks that can spread across all cloud providers at once.",
    "rationale": "Security issues are a bonus to the existing difficulties that come with the
implementation in the core logic.",
    "dependent_on": [
      "reasoning-17734"
    ],
    "pass": true,
    "evidence": "\u201cBlast radius: A bug or misconfiguration propagates to across all
Cloud providers simultaneously. Security exposure: Any vulnerability introduced into the
core logic is immediately shared across every provider.\u201d",
    "reasoning": "The golden answer explicitly states that modifying the core logic has a
large blast radius and highlights the security risk that vulnerabilities would spread
across all cloud providers."
  },
  {
    "id": "reasoning-18004",

```

```
"tag": "reasoning",
"weight": "penalty",
"description": "Claims that the Cluster Autoscaler does not read tags and depends on
a YAML file that is passed as a ConfigMap",
"rationale": "The Cluster Autoscaler exclusively uses tags to identify how and why to
scale a node group",
"dependent_on": [],
"pass": true,
"evidence": "The answer repeatedly emphasizes using tags on the Autoscaling Group
for configuration (e.g., \u201cFor AWS, this would be tags on the Autoscaling
Group\u201d). It never claims the Cluster Autoscaler does not read tags or relies on a
YAML ConfigMap instead.",
"reasoning": "Because the golden answer explicitly describes tag-based configuration
and does not assert that the Cluster Autoscaler depends on a YAML ConfigMap, it
avoids the penalized behavior."
},
{
  "id": "reasoning-17974",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies that `MinSize()` and `MaxSize()` in `aws_cloud_provider.go`
must be used to override the required functionality",
  "rationale": "This is existing place where the cluster decisions are made to set the min
and max size of the cluster",
  "dependent_on": [],
  "pass": true,
  "evidence": "There is explicit mention: \"In
`cluster-autoscaler/cloudprovider/aws/aws_cloud_provider.go` we need to override
`MinSize()` and `MaxSize()`.\"",
```

"reasoning": "The golden answer directly identifies the need to override `MinSize()` and `MaxSize()` within `aws\_cloud\_provider.go`, satisfying the criterion."

},

{

"id": "reasoning-17734",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies that per-cloud provider architecture modification is avoids touching the complex core logic avoiding code changes that require extensive testing",

"rationale": "Complex core logic changes can cause lot of side effects and burden to the maintainers maintaining the code because it requires the critical component to be stable and testing can become costly",

"dependent\_on": [],

"pass": true,

"evidence": "For these reasons, a per-cloud-provider implementation is strongly recommended over modifying the core logic.",

"reasoning": "The answer explicitly highlights that modifying core logic introduces broad risk and testing burden, and therefore recommends per-cloud provider architecture changes to avoid those issues, satisfying the criterion."

},

{

"id": "reasoning-18032",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that modes are not needed and that clusters must never hit their defaults if schedule tags are present",

"rationale": "While this is possible, defaults are set on a cluster after a lot of internal decisions. Min levels are there to optimise costs when the applications see the least traffic (like night) and this would cause lot of cost overruns if implemented wrong.",

"dependent\_on": [],

"pass": true,

"evidence": "The answer differentiates between a strict mode that enforces scheduled min/max values and a relaxed mode that allows fallback to defaults, e.g., \u201cIn a \u2018relaxed\u2019 mode \u2026 the CA can decide to fallback to the default value,\u201d and it never states that modes are unnecessary or that defaults must never be reached whenever schedule tags exist.",

"reasoning": "The golden answer explicitly uses modes and permits defaults in relaxed mode, so it does not make the penalized claim that modes are unnecessary or that defaults must never be hit when schedule tags are configured."

},

{

"id": "reasoning-17986",

"tag": "reasoning",

"weight": "bonus",

"description": "Identifies that `Refresh` is called once every one minute to refresh the tags making the config changes to schedule reflected when the refresh happens",

"rationale": "Refreshing of any new changes to the schedule config is important to capture new requirements for applications",

"dependent\_on": [],

"pass": true,

"evidence": "Refresh of the ASG for changes of schedules on the fly can happen when the `Refresh` function is called in

`cluster-autoscaler/cloudprovider/aws/aws\_manager.go`. This refreshes the ASG after a set interval, with the default being 1 minute.",

"reasoning": "The golden answer explicitly states that the Refresh function runs at a set interval with a default of one minute, showing awareness that schedule tag changes are picked up roughly every minute."

},

{

"id": "reasoning-17733",



```
"tag": "reasoning",
"weight": "critical",
"description": "Identifies that normal cluster autoscaler logic cannot handle periodic
bursts of traffic without eager configuration of thresholds or manual fallback",
"rationale": "This is the original use case that the answer has to identify to get to a
valid working solution and without this, the entire architecture falls apart",
"dependent_on": [],
"pass": true,
"evidence": "Normal Cluster Autoscaling logic is not enough to handle the above
scenarios without using some sort of manual fallback or highly eager configurations to
reduce thresholds. The wait time for the loop to run and detect that the metrics are
above the threshold is not enough...",
"reasoning": "The answer explicitly states that default cluster autoscaler logic cannot
handle predictable bursts without manual fallback or proactive threshold tuning,
satisfying the criterion."
},
{
  "id": "reasoning-17754",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies that `aws_manager.go` (or similar files in other cloud
providers) is the file where the extract logic for the new tag must be implemented",
  "rationale": "Initial read from ASG tags is required to load the configuration from AWS
to further process it",
  "dependent_on": [],
  "pass": true,
  "evidence": "In `cluster-autoscaler/cloudprovider/aws/aws_manager.go` add a new
extract function `extractScheduleFromAsg()` and wire it up to extract the schedule
format in `buildAsgFromAWS()` in
`cluster-autoscaler/cloudprovider/aws/auto_scaling_groups.go`.",
```

"reasoning": "The response explicitly identifies `aws\_manager.go` as the location to implement the extraction logic for the new tag, satisfying the criterion."

},

{

"id": "reasoning-17996",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that the current architecture of Cluster Autoscaler does not allow the implementation of any new Scheduling features",

"rationale": "This is wrong, the cluster autoscaler can be easily extended to have this feature",

"dependent\_on": [],

"pass": true,

"evidence": "The response explains that the Cluster Autoscaler wasn\u2019t originally designed for scheduled tasks but describes how to extend it via per-cloud-provider implementations, showing it can support new scheduling functionality.",

"reasoning": "The golden answer does not claim that the Cluster Autoscaler architecture prevents implementing new scheduling features; rather, it details how to extend it, so it avoids the prohibited claim."

},

{

"id": "reasoning-17745",

"tag": "reasoning",

"weight": "bonus",

"description": "Identifies the requirement of having a `strict` and `relaxed` mode for better handling of defaults",

"rationale": "Extra feature to handle defaults better, making the auto scaler use defaults when needed.",

"dependent\_on": [

```

    "completeness-17742"
  ],
  "pass": true,
  "evidence": "The answer explicitly defines configuration options including `strict` and `relaxed` flags: \u201c`strict` and `relaxed` are flags that can be used\u2026 In `relaxed` mode\u2026 but in `strict` mode\u2026 the autoscaler will not touch or reduce to the default config.\u201d It further details behavior of each mode under \u201cInner Workings.\u201d",
  "reasoning": "The response identifies and explains both `strict` and `relaxed` modes, showing how they manage defaults differently, satisfying the criterion."
},
{
  "id": "completeness-17981",
  "tag": "completeness",
  "weight": "critical",
  "description": "Gives a description of how `MinSize()` and `MaxSize()` can be implemented to support the cron and modes required.",
  "rationale": "Required to help the developer further investigate and implement the feature",
  "dependent_on": [
    "reasoning-17974"
  ],
  "pass": true,
  "evidence": "In the AWS-specific section, the answer states \"we need to override `MinSize()` and `MaxSize()`\" and later explains that in strict mode \"`MinSize()` returns 10, `MaxSize()` returns 40 regardless of pod demand\" while in relaxed mode \"`MinSize()` returns 2, `MaxSize()` returns 10 as soft suggestions.\",
  "reasoning": "The response explicitly describes how `MinSize()` and `MaxSize()` should be overridden and how they behave in both strict and relaxed cron-scheduled modes, satisfying the requirement."
}

```

```
},  
{  
  "id": "completeness-17714",  
  "tag": "completeness",  
  "weight": "bonus",  
  "description": "States that Cluster Autoscaler was originally not designed for time  
based autoscale usage",  
  "rationale": "Cluster Autoscaler was always designed to scale the cluster dynamically  
based on metrics, not to scale it based on specific set of time of day configurations",  
  "dependent_on": [],  
  "pass": true,  
  "evidence": "\u201cThe Kubernetes Cluster Autoscaler was originally not designed to  
handle scheduled tasks.\u201d",  
  "reasoning": "The golden answer explicitly states that the Cluster Autoscaler was not  
originally designed for scheduled tasks/time-based autoscaling, directly satisfying the  
criterion."  
},  
{  
  "id": "completeness-17966",  
  "tag": "completeness",  
  "weight": "critical",  
  "description": "States that `extractScheduleFromAsg` function can be implemented to  
handle the new tag",  
  "rationale": "New function is needed to keep the code modular and easy to replicate  
in other cloud providers",  
  "dependent_on": [  
    "reasoning-17754"  
  ],  
  "pass": true,
```

"evidence": "In `cluster-autoscaler/cloudprovider/aws/aws\_manager.go` add a new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule format in `buildAsgFromAWS()` in `cluster-autoscaler/cloudprovider/aws/auto\_scaling\_groups.go`. This new function captures the value with this key (one or many) and passes it to the function in `utils/schedule\_parser.go`.",

"reasoning": "The answer explicitly instructs to add the `extractScheduleFromAsg` function to handle the new tag by extracting schedule information and feeding it into the parser, satisfying the criterion."

},

{

"id": "completeness-17742",

"tag": "completeness",

"weight": "bonus",

"description": "States the sample scale up and down schedule config that can be used with AWS ASGs",

"rationale": "Example of how the config can be designed for use with AWS Auto Scaling Groups that will be used as tags",

"dependent\_on": [

"reasoning-17738"

],

"pass": true,

"evidence": "a. Scale Up and Maintain: [k8s.io/cluster-autoscaler/schedule](https://k8s.io/cluster-autoscaler/schedule): `0 8 \* \* \* min=10 max=40 strict`\n\n. Scale Down and Maintain: [k8s.io/cluster-autoscaler/schedule](https://k8s.io/cluster-autoscaler/schedule): `0 22 \* \* \* min=2 max=10 relaxed`\n",

"reasoning": "The response explicitly presents sample scale-up and scale-down schedule configurations as AWS Auto Scaling Group tags, satisfying the criterion."

},

{

"id": "completeness-17970",

```
"tag": "completeness",
"weight": "critical",
"description": "States that `buildAsgFromAWS` needs to be wired up to read the new
schedule tags",
"rationale": "New function needs to be called along with existing extract functions.",
"dependent_on": [
  "completeness-17966"
],
"pass": true,
"evidence": "\u201cIn `cluster-autoscaler/cloudprovider/aws/aws_manager.go` add a
new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule
format in `buildAsgFromAWS()` in
`cluster-autoscaler/cloudprovider/aws/auto_scaling_groups.go`. \u201d",
"reasoning": "The golden answer explicitly instructs wiring the new
schedule-extraction function into `buildAsgFromAWS`, satisfying the criterion."
},
{
  "id": "completeness-17748",
  "tag": "completeness",
  "weight": "critical",
  "description": "States that `utils/schedule_parser.go` can be used as a shared module
to help parse the schedules from all Cloud Providers",
  "rationale": "Standard way of parsing the schedules can help cloud provider
implementations to focus specifically on how to read the tags.",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cThe configuration parser can be implemented as a new file in
`utils/schedule_parser.go` that can handle all the parsing requirements. This can be a
shared module for all cloud providers to use. \u201d",
```

"reasoning": "The golden answer explicitly states that `utils/schedule\_parser.go` should be implemented as a shared module for all cloud providers, satisfying the criterion."

},

{

"id": "completeness-17715",

"tag": "completeness",

"weight": "bonus",

"description": "Gives an example of a use case in the real world from a tech company currently having the exact issue with Cluster Autoscaler",

"rationale": "Examples help the developer better understand if the feature was discussed before on public forums.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThis is evident further from Tech Blogs of various high scale apps like Hotstar and Netflix where live shows or sporting events attract a huge number of floating crowd. One such example is Hotstar handling a burst of traffic when a match starts and no longer needing that capacity when a match ends.\u201d",

"reasoning": "The golden answer cites real-world examples of Hotstar and Netflix facing the autoscaling timing issue, fulfilling the requirement to provide a concrete tech-company use case."

},

{

"id": "completeness-18019",

"tag": "completeness",

"weight": "penalty",

"description": "States that the implementation must go into vertical pod autoscaler because nodes scale when pods increase their CPU",

"rationale": "Nodes do scale when VPA increases CPU of a pod if required, but that does not answer why the schedule has to go there.",

```
"dependent_on": [],
"pass": true,
"evidence": "The golden answer discusses modifying the Kubernetes Cluster
Autoscaler with provider-specific schedules; it does not mention the Vertical Pod
Autoscaler or claim the implementation must go there.",
"reasoning": "Since the answer never states that the implementation must go into the
VPA, it avoids the specified penalty behavior."
},
{
  "id": "completeness-18025",
  "tag": "completeness",
  "weight": "penalty",
  "description": "Suggests to skip the existing `MinSize()` and `MaxSize()` functionality
by implementing new functions to handle this feature alone",
  "rationale": "Using existing functionality helps in reusing components like actually
calling AWS APIs to push the requirements to ASG controllers.",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201ccluster-autoscaler/cloudprovider/aws/aws_cloud_provider.go`
we need to override `MinSize()` and `MaxSize()`. \u201d",
  "reasoning": "The golden answer proposes using (overriding) the existing `MinSize()`
and `MaxSize()` methods rather than skipping them and creating entirely new
functionality, thus avoiding the penalized behavior."
},
{
  "id": "completeness-17988",
  "tag": "completeness",
  "weight": "bonus",
  "description": "States to manually restart the CA pod if the changes are required
instantly or near real time",
```



```
"rationale": "Near real time changes would prove costly if API calls made to AWS frequently and the best way is to restart the pod to force it into pulling ASG config.",
  "dependent_on": [
    "reasoning-17986"
  ],
  "pass": true,
  "evidence": "Restarting the CA Pod can also force refresh the ASG tags if the changes are required near real time.",
  "reasoning": "The golden answer explicitly instructs to manually restart the Cluster Autoscaler pod to force near real-time configuration refresh, satisfying the criterion."
},
{
  "id": "style-17990",
  "tag": "style",
  "weight": "bonus",
  "description": "Separates the answer into distinct sections that describe one part of the architecture at a time",
  "rationale": "Easier for the developer to follow the architecture and help in implementation planning",
  "dependent_on": [],
  "pass": true,
  "evidence": "The response is split into sections such as \"Core Logic Issues,\" \"Configuration Management,\" \"Tag Parsing,\" \"Inner Workings,\" and \"Conclusion,\" each addressing different parts of the architecture.",
  "reasoning": "These titled sections clearly separate the architectural aspects, making the structure easy to follow and satisfying the criterion."
},
{
  "id": "style-17991",
  "tag": "style",
```

```

    "weight": "bonus",
    "description": "Uses precise Go terminology to ground the architectural claims",
    "rationale": "Grounding the architectural claims on the language of the repository
helps developers to easily implement the required feature.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\u201cThe final struct\u2026`type scheduleConfig struct { \u2026 }`
\u2026 \u201cadd a new extract function `extractScheduleFromAsg()` \u2026 wire it up
\u2026 in `buildAsgFromAWS()` \u2026 override `MinSize()` and `MaxSize()`.\u201d",
    "reasoning": "The answer explicitly references Go constructs\u2014struct definitions
and function names\u2014and ties them to specific files in the repository, grounding the
architecture in precise Go terminology."
}
],
"passed": 23,
"failed": 0,
"total": 23,
"score_percent": 100.0,
"summary": "Passed 23/23 criteria (100.0%)"
}

```

## Test Model

```
=====
```

RUN 1/3

```
=====
```

Below is an architecture-level plan for adding “time-of-day autoscaling” to Cluster Autoscaler (CA). It focuses on where to extend the codebase, which new abstractions to introduce, and how to expose/configure the feature so that we can move into implementation with minimal churn later.

## ## 1. Desired behaviour

- \* Allow operators to define time-based rules (e.g., weekdays 08:00-18:00) that change the expected capacity of specific node groups without waiting for workload-driven scale-up.
- \* Rules should raise the *minimum* capacity (pre-warm for predictable peaks) and optionally lower the *maximum* capacity (to force scale-down after the window).
- \* Rules must be cluster-safe: they should honour regular CA safeguards (max total nodes, resource limits, etc.) and integrate with existing events/metrics.

## ## 2. Configuration surface

### ### 2.1 CLI / flags (`cluster-autoscaler/main.go`)

Add gated support so the feature is opt-in:

```
``go
var (
    scheduledScalingEnabled = flag.Bool("scheduled-scaling-enabled", false, "Enable
time-of-day scaling rules")
    scheduledScalingConfigMap = flag.String("scheduled-scaling-configmap",
"cluster-autoscaler-schedule", "ConfigMap containing schedule definitions")
    scheduledScalingNamespace = flag.String("scheduled-scaling-namespace", "",
"Namespace for the schedule ConfigMap (defaults to --namespace)")
    scheduledScalingDefaultTZ = flag.String("scheduled-scaling-timezone", "UTC",
"Default timezone for schedule windows")

```

```
    scheduledScalingLookahead = flag.Duration("scheduled-scaling-lookahead",
30*time.Minute, "How far ahead CA should protect capacity before a window begins")
)
``
```

Propagate these knobs into `AutoscalingOptions` (`config/autoscaling\_options.go`) and `AutoscalingContext` (`context/autoscaling\_context.go`) so downstream processors can consume them.

### ### 2.2 ConfigMap data model

Store schedules in a dedicated ConfigMap (similar to the priority expander config in `expander/priority`). Example:

```
``yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-autoscaler-schedule
  namespace: kube-system
data:
  schedule.yaml: |
    timezone: America/Los_Angeles # optional default
    rules:
      - name: business-hours
        days: [Mon, Tue, Wed, Thu, Fri]
        start: "08:00"
        end: "18:00"
        nodeGroups:
          - id: "ng-us-west-1"
```

```
    min: 20
    max: 100
- name: nightly-batch
  days: [Mon, Tue, Wed, Thu, Fri]
  start: "21:00"
  end: "23:00"
  nodeGroups:
  - id: "ng-batch"
    min: 40
...

```

\* `days` is optional; absence  $\Rightarrow$  7 days.

\* `start`/`end` are local times in HH:MM (24h).

\* `min`/`max` overrides are independent (max optional).

We can extend this later with cron expressions or per-rule timezones if needed.

---

## ## 3. New internal components

### ### 3.1 Schedule loader & cache (new package `cluster-autoscaler/scheduledscaling`)

Responsibilities:

1. Watch/read the ConfigMap via the existing lister framework (`utils/kubernetes/listers.go`) to avoid bespoke clients.
2. Parse YAML into canonical structs, validate overlaps/conflicts, convert to UTC windows.
3. Expose a pure function:

```

``go
type Overrides struct {
    NodeGroup string
    MinOverride *int
    MaxOverride *int
    Active bool      // true if current time within window
    NextWindow time.Time // start of next window (for lookahead)
}

type Planner interface {
    CurrentOverrides(now time.Time) []Overrides
    Refresh(ctx context.Context) // triggered when config map changes
}
...

```

Implementation details:

- \* On ConfigMap changes, rebuild an in-memory schedule map keyed by node group.
- \* Accept a `Clock` interface to keep tests deterministic.

### ### 3.2 Processor hook

Introduce a new processor (e.g., `ScheduledScalingProcessor`) so we stay within the existing extension framework (`cluster-autoscaler/processors`). Add it to `AutoscalingProcessors` in `processors/processors.go` and gate its creation in `core/autoscaler.go` when the feature flag is set.

Responsibilities per RunOnce:

1. Inspect `clusterstate.ClusterStateRegistry` (`ClusterStateRegistry.GetStatus`) for current/target sizes.

2. Ask the `Planner` for overrides at `currentTime`.

3. Enforce **min overrides**:

- \* If `currentTarget < minOverride`, calculate `delta`.

- \* Use the same path as scale-up (`core/scale\_up.go`) by preparing a fake “demand” object rather than bypassing CA safeguards.

- \* Implementation sketch:

- Add helper `ScheduleScaleUp(context, registry, nodeInfos, nodeGroupID, delta)` that wraps `ScaleUp` internal primitives (choose node group, call `nodeGroup.IncreaseSize`, register requests, emit events).

- Because there are no backing pods, log events with a dedicated reason (`ScheduledScaleUp`) and skip predicate simulations (the node template is already known).

4. Respect **max overrides** / future windows:

- \* Before scale-down, reduce the “acceptable range” for a node group so CA will hurry scale-down when `currentTarget > maxOverride`.

- \* Implementation: extend `ClusterStateRegistry.updateAcceptableRanges` to subtract scheduled deltas. This ensures scale-down logic sees those nodes as removable and still honours graceful policies.

5. “Lookahead” protection:

- \* If a window starts within `--scheduled-scaling-lookahead`, treat `minOverride` as active already to give CA time to spin nodes up.

Integration point in `core/static\_autoscaler.go`:

```
...
```

```
stateUpdateStart := time.Now()
```

```
...
```

```
if a.scheduledScalingProcessor != nil {
```

```
    if err := a.scheduledScalingProcessor.Apply(autoscalingContext,  
a.clusterStateRegistry, nodeInfosForGroups, currentTime); err != nil {
```

```

    klog.Errorf("Scheduled scaling failed: %v", err)
    return errors.ToAutoscalerError(errors.InternalError, err)
}
}
...

```

Place this **after** `updateClusterState` (so we have fresh readiness info) and **before** evaluating unschedulable pods.

### ### 3.3 Cluster state awareness

`ClusterStateRegistry` already tracks readiness/target sizes per node group (`perNodeGroupReadiness`, `acceptableRanges`). We need two small extensions:

1. Provide a helper `GetNodeGroupSizes(id string) (ready, target int)` so the processor can decide how many nodes to add/remove.
2. Allow temporary min/max overrides when computing acceptable ranges. Add two optional maps (`scheduledMin`, `scheduledMax`) injected each loop by the processor, and apply them inside `updateAcceptableRanges`. This prevents scale-down from going below scheduled mins even if there is no scheduled scale-up activity that iteration.

All new data should live only in memory (no API changes) and reset each loop to avoid stale overrides if the feature is disabled at runtime.

---

### ## 4. Control flow summary

1. **Startup**:
  - \* Flags parsed in `main.go`.
  - \* `createAutoscalingOptions` populates new fields.



- \* `core.NewAutoscaler`` wires a `ScheduledScalingProcessor`` only when enabled.

2. **RunOnce** (after nodes/pods fetched):

- 1. `ScheduledScalingProcessor.Apply()` obtains overrides from the planner.

- 2. For each node group:

- \* If `CurrentTarget < ScheduledMin``, register a scale-up (calls into the same code path used for pod-driven scale-ups).

- \* If `CurrentTarget > ScheduledMax``, push info into `ClusterStateRegistry`` so the forthcoming scale-down run immediately considers those nodes as removable.

- 3. Downstream logic (filter pods, ScaleUp, ScaleDown) proceeds unchanged but sees the cluster already converging to the scheduled bounds.

3. **Config updates**: the planner listens for ConfigMap changes; when they happen, it rebuilds rules and toggles a `generation`` counter. The processor checks the counter each loop to decide whether to log events (`"Reloaded scheduled scaling config rev=123"`).

---

## ## 5. Validation & safety

- \* **Conflict handling**: When overlapping windows specify different mins for the same node group, resolve deterministically (e.g., "highest min wins"; log a warning).

- \* **Timezone correctness**: convert rules to UTC at load time; support per-rule `timezone`` overrides (optional field) falling back to `spec.timezone`` or flag default.

- \* **Back-pressure**: honour `MaxNodesTotal`` / resource limits by verifying deltas against `config.AutoscalingOptions`` before calling `IncreaseSize``. If a scheduled request would violate limits, emit a warning event and skip the action.

- \* **Observability**:

- \* Emit events (`ScheduledScaleUp``, `ScheduledScaleDown``) on the ConfigMap owner reference (or the CA Deployment) so operators can audit behaviour.

\* Add metrics: ``scheduled_scale_up_requested_total``, ``scheduled_override_active{node_group}`` gauge, etc., alongside the existing metrics pipeline (``metrics`` package).

---

## ## 6. Testing strategy

- \* Unit tests for the planner (parse schedules, timezone math, overlap resolution).
- \* Unit tests for processor logic using fake node groups (``utils/test``) to verify scale-up/down decisions.
- \* Integration-style test around ``StaticAutoscaler.RunOnce`` that injects a fake planner and asserts CA registers the correct scale-up requests without actual unschedulable pods.

---

## ## 7. File touchpoints recap

| Area | Files |

|-----|-----|

| Flags & options | ``cluster-autoscaler/main.go``, ``config/autoscaling_options.go``,  
``context/autoscaling_context.go`` |

| Processor wiring | ``core/autoscaler.go``, ``processors/processors.go``, new  
``processors/scheduledscaling/...`` |

| Core loop integration | ``core/static_autoscaler.go`` |

| Cluster state support | ``clusterstate/clusterstate.go`` (helpers & acceptable-range  
overrides) |

| Planner package | new ``cluster-autoscaler/scheduledscaling/{loader.go, planner.go}``  
plus tests |

| Docs | `cluster-autoscaler/README.md`, `cluster-autoscaler/FAQ.md` (new section describing scheduled scaling) |

---

This architecture keeps the feature isolated (new planner + processor), reuses existing CA mechanics for actual scaling, and leans on ConfigMaps for operator ergonomics. Once you're comfortable with the design, we can start slicing the work into implementation PRs (flags/options, planner, processor, tests, docs).

=====

RUN 2/3

=====

### ### 1. Executive summary

We can add "time of day" autoscaling without refactoring the entire Cluster Autoscaler (CA) loop by introducing a **scheduled-scaling engine** that (a) periodically computes time-based capacity overrides and (b) feeds those overrides into existing scale-up and scale-down paths. The integration hinges on a few well-scoped entry points:

| Area | File (line range) | Why it matters |

|----|-----|-----|

| CLI / options | `cluster-autoscaler/main.go:L82-L240` | All new flags and initial parsing happen here. |

| Config object | `cluster-autoscaler/config/autoscaling\_options.go` |

`AutoscalingOptions` must carry schedule data so that it reaches the core logic. |

| Context | `cluster-autoscaler/context/autoscaling\_context.go` | Every processor/scale component reads configuration from here. |

| Main loop | `cluster-autoscaler/core/static\_autoscaler.go` | Best place to inject schedule enforcement before regular scale-up/down logic. |

| Scale-down candidate filtering |

`cluster-autoscaler/processors/nodes/pre\_filtering\_processor.go:L31-L74` | Today it only respects static min-size. We can reuse this choke point to honor time-based minimums. |

## ### 2. Configuration & API design

### 1. \*\*New CLI flags (parse in `main.go`)\*\*

- `--scheduled-scaling-configmap=<ns/name>` (mutually exclusive with `--scheduled-scaling-config-file`).
- `--scheduled-scaling-default-timezone=<tz>` (fallback when a window does not specify one).
- Optional `--scheduled-scaling-dry-run` to emit metrics/events without actually touching node groups.

### 2. \*\*File / ConfigMap schema (YAML)\*\*

```yaml

policies:

- name: "weekday-daytime"  
nodeGroupSelector: # match either explicit nodegroup IDs or labels  
ids: ["gke-primary", "gke-batch"]  
windows:
  - days: ["Mon","Tue","Wed","Thu","Fri"]  
start: "08:30"  
end: "18:00"  
timezone: "America/Los\_Angeles"  
minNodes: 10  
maxNodes: 40 # optional upper guard  
targetCapacity: 25 # optional proactive scale-up  
scaleDownDelay: "30m" # don't allow scale-down for 30m after window closes

...

Parsing/validation lives in a new package (e.g. ``cluster-autoscaler/scheduled/config.go``). It should:

- \* Compile windows into absolute ranges (``time.Time`` with week offsets).
- \* Resolve timezones once (use ``time.LoadLocation``).
- \* Support overlapping policies by taking the **max** of all requested mins for a node group.

### 3. **AutoscalingOptions additions**

In ``config/autoscaling_options.go``, add:

```
``go
ScheduledScalingEnabled bool
ScheduledScalingPolicies []scheduled.Policy
ScheduledScalingDefaultTZ string
...
```

Update ``createAutoscalingOptions()`` (``main.go:L176+``) to populate these fields after reading the flag(s). When the source is a `ConfigMap`, reuse existing client-go config reading (``config/kubernetes.go``) machinery.

### 4. **Context plumbing**

Add ``ScheduleEngine scheduled.Engine`` to ``context.AutoscalingContext``. The engine is constructed inside ``core.NewAutoscaler()`` once options are validated. Every subsystem can then call ``ctx.ScheduleEngine`` instead of re-reading files.

### ### 3. Scheduled scaling engine (new pkg ``cluster-autoscaler/scheduled``)

Responsibilities:

1. Hold compiled policies.
2. Expose pure functions:
  - ``DesiredMin(nodeGroupID string, when time.Time) (min int, ok bool)``

- `DesiredTarget(nodeGroupID string, when time.Time) (target int, reason string, ok bool)`

- `ScaleDownCooldown(nodeGroupID string, when time.Time) time.Time`

3. Optionally watch the ConfigMap (using the same client set created in `context.NewAutoscalingKubeClients`). When data changes, atomically swap the compiled schedule.

Implementation sketch:

```
```go
```

```
type Engine interface {
```

```
    Evaluate(now time.Time) map[string]Evaluation // keyed by nodeGroup ID
```

```
}
```

```
type Evaluation struct {
```

```
    MinNodes      *int
```

```
    TargetNodes    *int
```

```
    MaxNodes      *int
```

```
    ScaleDownBlockedUntil *time.Time
```

```
}
```

```
```
```

### 4. Control-loop integration (core/static\_autoscaler.go)

1. **Early enforcement (pre scale-up)**

At the beginning of `RunOnce` (~ line 215), after `nodeInfosForGroups` is built but before we look at unschedulable pods, invoke:

```
```go
```

```
a.applyScheduledScaleUp(currentTime, nodeInfosForGroups)
```

```
```
```

Algorithm:

- For each node group with a `TargetNodes`` override greater than the current target (obtain current target via `clusterStateRegistry``'s acceptable range or `NodeGroup.TargetSize()``):
  - \* Call `NodeGroup.IncreaseSize(delta)`` (same path as regular scale-up).
  - \* Mirror what `ScaleUp()`` does: `clusterStateRegistry.RegisterOrUpdateScaleUp()`` so health/backoff logic stays consistent.
  - \* Emit metric `scheduled_scale_up_forced`` with labels (policy, nodegroup).
- For groups below their **minimum** override, treat `TargetNodes = MinNodes`` so they get bumped immediately even if no pods are pending.

## 2. **Scale-down guard**

When `ScaleDown`` asks the node processor for candidates (`processors/nodes/pre_filtering_processor.go:L31-L74``), compute an **effective min**:

```
``go
scheduledMin := ctx.ScheduleEngine.MinNodes(nodeGroup.Id(), now)
enforcedMin := max(nodeGroup.MinSize(), scheduledMin)
if size <= enforcedMin { skip node }
``
```

This single change prevents draining nodes that would violate the next window. Because this file already talks to `AutoscalingContext``, no extra plumbing is required beyond adding the engine to the context.

## 3. **Cooldown handling**

Before calling `scaleDown.TryToScaleDown`` (~line 500), check whether any node group has `ScaleDownBlockedUntil > now`` and, if so, set `scaleDownStatus.Result = status.ScaleDownInCooldown``. You can reuse the existing callback mechanism (`processorCallbacks``) by setting `SetExtraValue("scheduled-scale-down-blocked", true)`` so downstream processors can emit events.

## 4. **Cluster-state awareness**

To keep `clusterstate.ClusterStateRegistry`` (see `cluster-autoscaler/clusterstate/clusterstate.go``) honest, feed scheduled minimums into `updateAcceptableRanges()``:

- After computing `MinNodes := size - readiness.LongUnregistered``, wrap it with `max(MinNodes, scheduledMin)``.

This ensures health checks and status ConfigMaps reflect the enforced minimum.

### ### 5. Metrics, events, and observability

Add gauges/counters in `metrics/metrics.go``:

- `scheduled_scale_forced_up_total`` (labels: policy, nodegroup, reason=min|target).
- `scheduled_scale_blocked_down`` gauge (per nodegroup) to show cooldowns.

Emit EVENTS (via `ctx.LogRecorder``) when:

- A schedule bumps capacity even though no pods were pending (“ScheduledScalingUp”).
- Scale-down is skipped due to upcoming window (“ScheduledScalingGuard”).

### ### 6. Config reload & validation

- When using ConfigMap, reuse the same `ListerRegistry`` informer factory (`context/autoscaling_context.go``). Add a lightweight controller under `scheduled/loader.go`` that watches that ConfigMap, parses YAML, and updates the engine through an atomic pointer.
- Reject invalid configs during boot (fail fast in `main.go``).
- Surface validation errors through `/health-check`` by wiring them into `metrics.HealthCheck`` (already instantiated in `main.go:L356``).

### ### 7. Failure / edge cases



- **Overlapping policies**: define deterministic merge order (e.g., `max(minNodes)`, `min(maxNodes)`, highest target wins). Document this in the YAML schema.
- **Timezones & DST**: compile windows using `time.Location``, and store next-trigger timestamps to avoid recomputing every loop.
- **CloudProvider constraints**: if a scheduled target exceeds `NodeGroup.MaxSize()``, emit warning and cap at max. If it is lower than `MinSize()``, still allow `MinSize()`` (the provider already enforces that).
- **Dry run**: honor `--scheduled-scaling-dry-run`` by skipping the `IncreaseSize`` call but still logging what would happen.

### ### 8. Summary of code touch points

| Change | Files |

|-----|-----|

| Flags + option wiring | `cluster-autoscaler/main.go`` |

| Options struct & serialization | `cluster-autoscaler/config/autoscaling_options.go`` |

| New scheduled package | `cluster-autoscaler/scheduled/*`` |

| Context carries engine | `cluster-autoscaler/context/autoscaling_context.go`` |

| Static autoscaler invokes engine | `cluster-autoscaler/core/static_autoscaler.go`` |

| Scale-down candidate filter respects schedule |

`cluster-autoscaler/processors/nodes/pre_filtering_processor.go`` |

| Cluster state min-range incorporates schedule |

`cluster-autoscaler/clusterstate/clusterstate.go`` |

| Metrics | `cluster-autoscaler/metrics/metrics.go`` |

With this architecture, the existing CA behavior remains unchanged when no schedule is configured, but operators gain precise, time-of-day control over minimum/target capacity via declarative policies.

=====

RUN 3/3

=====

### ### 1. Current control loop and places we must touch

| Responsibility | Where it lives today | Why it matters for time-of-day scaling |

| -- | -- | -- |

| CLI / flag parsing → `AutoscalingOptions` | `cluster-autoscaler/main.go` (flag definitions around lines 70-250) feeds into `createAutoscalingOptions()` which populates `config.AutoscalingOptions` (`config/autoscaling\_options.go`) | Any new feature must be surfaced here (flags, config files, ConfigMaps) so that the options are visible everywhere through `AutoscalingContext`. |

| Main reconciliation loop | `core/static\_autoscaler.go`, especially  
`(\*StaticAutoscaler).RunOnce()` (roughly lines 120-520) | This loop decides whether to scale up/down. Time-based logic needs to run inside this loop so it can influence both scale-up triggers and how scale-down treats nodes. |

| Scale-down protection | Also in `core/static\_autoscaler.go` (lines 530-880) and  
`core/scale\_down.go` | If a schedule requires “keep X nodes from Y:00 to Z:00”, scale-down must respect that floor (e.g., mark nodes as unremovable or adjust min-size logic). |

| Cloud provider interaction | NodeGroup abstractions in `cloudprovider` packages |

Enforcing scheduled min/max per node group eventually calls

`IncreaseSize/DecreaseTargetSize`, so any direct manipulations must go through these interfaces. |

### ### 2. Proposed architecture for time-of-day autoscaling

#### #### 2.1 Configuration model

Add a structured policy to `AutoscalingOptions`:

```go

```
// config/timebased.go (new file)
type TimeWindow struct {
    Start string `json:"start" // "22:00"
    End   string `json:"end"  // "06:00"
    Min   int    `json:"min"`
    Max   *int    `json:"max,omitempty"`
    Days  []time.Weekday `json:"days,omitempty" // optional
    Timezone string `json:"timezone,omitempty"`
}

type TimeBasedPolicy struct {
    NodeGroupID string    `json:"nodeGroupId,omitempty"`
    NodeGroupSelector map[string]string `json:"nodeGroupSelector,omitempty" // match
labels/tags
    Windows []TimeWindow `json:"windows"`
}

type TimeBasedAutoscalingConfig struct {
    Policies []TimeBasedPolicy `json:"policies"`
}
...

```

Then extend `config.AutoscalingOptions` (`config/autoscaling\_options.go`) with:

```
``go
    TimeBasedConfigPath string
    TimeBasedConfig    *TimeBasedAutoscalingConfig
...

```

New CLI flags in `main.go` (near other flag declarations):

```

...
timeBasedConfigFile = flag.String("time-based-config", "",
    "Path to JSON/YAML describing time-of-day scaling policies")
timeBasedConfigMap = flag.String("time-based-configmap", "",
    "ConfigMap (namespace/name) to watch for time-of-day policies")
timeBasedDefaultTZ = flag.String("time-based-default-timezone", "UTC",
    "Default IANA timezone for time-of-day rules")
...

```

During `createAutoscalingOptions()` parse the file/ConfigMap (re-using the existing kube client, similar to how dynamic configs are handled under `config/dynamic`). We already write/read ConfigMaps in `cluster-autoscaler/config/dynamic`, so we can mirror that pattern.

**\*Why both file and ConfigMap?\*** – File works for static deployments; ConfigMap enables live reconfiguration without restart.

## #### 2.2 Runtime wiring

1. **\*\*AutoscalingContext exposure\*\*** – add a helper in `context/autoscaling_context.go` (not shown above but fed by options) to cache the parsed `TimeBasedAutoscalingConfig` and to expose a method:

```

```go
func (c *AutoscalingContext) ActiveTimePolicies(now time.Time) []ResolvedPolicy
...

```

`ResolvedPolicy` would be the concrete “for nodeGroup <id>, enforce min=m, max=n right now”.

2. **New processor** – mimic existing processor slots (see `processors`` package).  
Create `processors/timebased/time_based_processor.go`` with interface:

```
``go
type TimeBasedProcessor interface {
    EnsureScheduledCapacity(ctx *context.AutoscalingContext, now time.Time,
        allNodes []*apiv1.Node, nodeInfos map[string]*scheduling.NodeInfo)
    (actions ActionsTaken, err error)
}
``
```

Default implementation:

- Iterate active policies.
- Resolve target node groups (by ID or via selector using `cloudProvider.NodeGroups()`` metadata).
- Compare desired min/max with:
  - Actual size (`nodeGroup.TargetSize()``).
  - Kubernetes view (length of nodes belonging to that group).
- If current < scheduled min:
  - Call `nodeGroup.IncreaseSize(delta)`` and record the action into `clusterStateRegistry.RegisterOrUpdateScaleUp(...)`` so the registry is aware (see `core/static_autoscaler.go`` lines 299-333 for similar flows).
  - Optionally annotate nodes to prevent scale-down until the scheduled period ends.
- If current > scheduled max (if defined):
  - Mark extra nodes as “scheduled for removal” by injecting an artificial unremovable reason except for the nodes above max, and let scale-down handle them in the next phase (or call `nodeGroup.DecreaseTargetSize`` directly if allowed).

Hook this processor from `core.NewAutoscaler()` (right after existing processors are wired in `main.go`, lines ~280-307). If no config is provided, use a no-op implementation.

3. **RunOnce integration** – early in `StaticAutoscaler.RunOnce()` after we've collected node lists and before we branch into scale-up/down logic (roughly after we call `a.updateClusterState(...)` around line 274), invoke:

```
```go
if a.processors != nil && a.processors.TimeBasedProcessor != nil {
    actions, err := a.processors.TimeBasedProcessor.EnsureScheduledCapacity(
        a.AutoscalingContext, currentTime, allNodes, nodeInfosForGroups)
    ...
}
```
```

`actions` can annotate the `autoscaler` status ConfigMap (already written in the `defer` block later) and record metrics.

#### 4. **Scale-down awareness**

- For nodes that are needed only to satisfy a schedule, we must not let regular utilization-based logic evict them. We can achieve this by extending `TimeBasedProcessor` to mark nodes with the existing annotation [cluster-autoscaler.kubernetes.io/scale-down-disabled](https://cluster-autoscaler.kubernetes.io/scale-down-disabled) (see `ScaleDownDisabledKey` in `core/scale_down.go` line 35). A helper can:

- Choose `min` nodes in the group (e.g., oldest first or ones already annotated) and ensure the annotation is set.
- When the window ends, remove the annotation so normal scale-down resumes.

- Because ``scale_down.go`` already checks for that annotation (``hasNoScaleDownAnnotation``, lines 615-626), no further changes are required there besides ensuring annotations are managed.

5. **Max enforcement** (optional): to treat “scheduled max”, feed a synthetic resource limit into ``scaleDownResourcesLeft`` so that the node removal side prefers those nodes even if utilization is low. This could be as simple as storing “policyMax[nodeGroupID]” inside ``staticAutoscalerProcessorCallbacks`` (``core/static_autoscaler.go``, lines 90-115) and reading it in ``ScaleDown.chooseCandidates``. For a first iteration, it’s usually enough to request ``DecreaseTargetSize`` via cloud provider if current > scheduled max.

### #### 2.3 Data flow summary

...

Flags/ConfigMap -> AutoscalingOptions.TimeBasedConfig

↓

context.AutoscalingContext (new helper)

↓

processors.TimeBasedProcessor.EnsureScheduledCapacity()

↓

- Direct NodeGroup Increase/Decrease calls
- Node annotations for scale-down protection

↓

ScaleUp/ScaleDown logic sees updated cluster state (some nodes already pending, some protected)

...

### ### 3. Configuration mechanisms

1. **Static file** – mount JSON/YAML via ConfigMap/Secret, pass ``--time-based-config=/etc/ca/schedule.yaml``.

2. **Live ConfigMap** – set `--time-based-configmap=kube-system/ca-time-schedule`. Use an informer (similar to `utils/kubernetes/listers.go`) to watch for updates; when `resourceVersion` changes, re-parse and swap the in-memory policy (guard with `RWMutex`). Because `AutoscalingContext` already has kube clients, the watcher can be started alongside other background components in `StaticAutoscaler.Start()`.

Example YAML:

```
``yaml
policies:
- nodeGroupSelector:
  cloud.google.com/node-group: spot-ng
  windows:
  - start: "22:00"
    end: "06:00"
    min: 10
    timezone: "America/Los_Angeles"
  - start: "06:00"
    end: "22:00"
    min: 3
- nodeId: "prod-ng-a"
  windows:
  - start: "00:00"
    end: "24:00"
    min: 5
    max: 15
...

```

Validation should happen on load (reject overlapping windows per policy, ensure `min <= max`, parse timezone).



### ### 4. Edge cases & considerations

- **Time zones / DST** – use `time.Location` when parsing windows. Decide how to handle overnight spans (e.g., start 22:00, end 06:00) by normalizing to “clock range possibly wrapping past midnight.”
- **Overlapping policies** – define deterministic precedence (e.g., most specific selector > ID > default). Document this.
- **Interaction with HPA / regular CA logic** – scheduled min guarantees capacity but does not suppress reactive scale-up; unschedulable pods still trigger scale-up beyond scheduled min. Conversely, scale-down never goes below scheduled min even if utilization is low.
- **Metrics & status** – extend `metrics` to record when schedule-driven scale-ups/downs happen and surface the active window in the status ConfigMap (written near the `defer` block in `RunOnce`).
- **Testing** – add unit tests for policy parsing, matching, and a fake clock integration test that runs `RunOnce` at different times to ensure the processor requests the right actions. Use the existing fake cloud provider in `cluster-autoscaler/core/scale_test_common.go`.

### ### 5. Implementation roadmap

#### 1. **Config parsing**

- Add new struct definitions + parser (`config/timebased.go`).
- Extend CLI flags and `createAutoscalingOptions()` (`main.go` and `config/autoscaling_options.go`).

#### 2. **Context plumbing**

- Store parsed config in `AutoscalingContext`; expose helper methods.

#### 3. **Processor**

- Implement `TimeBasedProcessor` (new package under `processors/timebased`).
- Wire into `ca_processors.DefaultProcessors()` (probably in `processors/processors.go`) and `core.NewAutoscaler()` so it's available via `a.processors.TimeBasedProcessor`.

#### 4. **Run loop integration**

- Call processor early in `RunOnce`.
- Manage annotations and NodeGroup size changes inside the processor.

#### 5. **Optional config reloader**

- If ConfigMap-based config is desired, create a lightweight watcher using existing informer utilities in `utils/kubernetes/listers.go`.

With this layout the feature stays orthogonal to existing scale-up/down logic, leverages the Processor abstraction, and keeps configuration centralized in `AutoscalingOptions`, making it easy to reason about and test.

### **model evaluation**

```
{  
  "points": 7.0,  
  "maxPoints": 36.0,  
  "score_percent_weight": 19,  
  "summary_weight": "7.0/36.0 points (19%)",  
  "eval_runs": 3,  
  "eval_runs_succeeded": 3,  
  "per_run_points": [  
    7.0,  
    7.0,  
    7.0  
  ],  
}
```

```
"results": [  
  {  
    "id": "reasoning-17738",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-17737",  
    "weight": "bonus",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-18004",  
    "weight": "penalty",  
    "pass": true,  
    "points": -1.0  
  },  
  {  
    "id": "reasoning-17974",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-17734",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  }  
]
```

```
},  
{  
  "id": "reasoning-18032",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17986",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17733",  
  "weight": "critical",  
  "pass": true,  
  "points": 3.0  
},  
{  
  "id": "reasoning-17754",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17996",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0
```

```
},  
{  
  "id": "reasoning-17745",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17981",  
  "weight": "critical",  
  "pass": true,  
  "points": 3.0  
},  
{  
  "id": "completeness-17714",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17966",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17742",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0
```

```
},  
{  
  "id": "completeness-17970",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17748",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17715",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-18019",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-18025",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0
```

```
},  
{  
  "id": "completeness-17988",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "style-17990",  
  "weight": "bonus",  
  "pass": true,  
  "points": 1.0  
},  
{  
  "id": "style-17991",  
  "weight": "bonus",  
  "pass": true,  
  "points": 1.0  
}  
]  
}
```

model eval score    19.4

difficulty        **hard**